



Memoria Descriptiva

HabitHero

Version 1.0.0

1. Problema

Entre los 8 y los 12 años, los niños empiezan a asumir rutinas propias (higiene, orden, estudio, descanso), pero todavía no tienen la noción de continuidad que hace que un hábito se sostenga. En la práctica, la constancia la sostiene un adulto que recuerda, insiste y a veces regaña, lo cual convierte la rutina en un conflicto y hace que el niño no se apropie de ella.

Las herramientas habituales no encajan con esta edad:

- Las agendas y listas de tareas para adultos son densas y exigen leer y escribir mucho.
- Las apps de hábitos del mercado suelen requerir cuenta, conexión y publicidad, y con frecuencia usan mecánicas de castigo (rachas que se pierden, puntuaciones negativas, recordatorios que culpabilizan).
- Muchas recogen datos personales, lo cual es especialmente delicado con menores.

Falta una herramienta que el niño pueda usar solo, que le devuelva una señal visible de su propio avance, y que no dependa de internet ni de datos personales.

2. Objetivos

Objetivo general

Construir una aplicación Android nativa que ayude a niños de 8 a 12 años a desarrollar hábitos diarios mediante marcación visual, rachas, monedas, niveles e insignias, funcionando por completo sin conexión.

Objetivos específicos

1. Permitir marcar hábitos con un solo toque, una vez por día, y deshacer una marca puesta por error.
2. Mostrar el progreso de forma visual: racha actual, mejor racha y los últimos siete días, agrupado por categoría.
3. Motivar con recompensas acumulativas (monedas, experiencia, niveles), al menos cinco insignias, un desafío sorpresa diario y una tienda donde gastar las monedas.
4. Permitir crear hábitos propios con nombre, icono, color, categoría, días de la semana y un recordatorio local opcional.
5. Guardar toda la información en el dispositivo con Room, sin cuentas ni servidores.
6. Mantener un tono siempre positivo: nunca castigos, reproches ni pérdidas.
7. Automatizar pruebas, compilación y publicación del APK mediante GitHub Actions.

3. Alcance

Incluido

- Siete pantallas: bienvenida, inicio, crear/editar hábito, progreso, tienda, insignias y ajustes.
- Seis hábitos predeterminados listos para usar, cada uno con su categoría.
- Hábitos personalizados con nombre de hasta 30 caracteres, icono, color, categoría, días y recordatorio local opcional.
- Marcado y desmarcado con una marca máxima por hábito y día.
- Racha actual y mejor racha por hábito.
- Monedas, experiencia y niveles.
- Siete insignias con progreso visible.
- Un desafío sorpresa distinto cada día, con recompensa propia.
- Una tienda para gastar monedas en avatares y marcos nuevos.
- Un avatar que gana pequeños adornos visuales según el nivel alcanzado.
- Recordatorios locales opcionales por hábito, con permiso de notificaciones también opcional.
- Filtro de hábitos por categoría y resumen de progreso agrupado por categoría.
- Resumen visual de los últimos siete días.
- Reinicio total de datos con doble confirmación.
- Datos semilla para poder probar la aplicación desde el primer arranque.
- Interfaz completa en español.

Excluido deliberadamente

Registro, login, servidor, API externa, Firebase, publicidad, compras reales dentro de la app, chat, mapas, analítica, rastreo y acceso a internet. Los únicos permisos declarados (`POST_NOTIFICATIONS` y `RECEIVE_BOOT_COMPLETED`) existen solo para el recordatorio local opcional descrito en §7 y no dan acceso a la red ni a datos del usuario.

Fuera del alcance de esta versión

Varios perfiles en un mismo dispositivo, exportación o copia manual de datos, modo oscuro, y estadísticas de más de siete días.

4. Requisitos

4.1 Requisitos funcionales

ID	Requisito
RF-01	Elegir avatar y nombre en la primera apertura.

ID	Requisito
RF-02	Mostrar en Inicio solo los hábitos programados para hoy, con filtro opcional por categoría.
RF-03	Marcar un hábito con un toque.
RF-04	Impedir marcar el mismo hábito dos veces el mismo día.
RF-05	Desmarcar una actividad marcada por error, con confirmación.
RF-06	Crear un hábito con nombre (≤ 30 caracteres), icono, color, categoría, días y recordatorio opcional.
RF-07	Editar y borrar un hábito existente.
RF-08	Calcular y mostrar la racha actual de cada hábito.
RF-09	Calcular y mostrar la mejor racha de cada hábito.
RF-10	Otorgar monedas y experiencia al marcar.
RF-11	Subir de nivel al acumular experiencia.
RF-12	Ofrecer al menos cinco insignias con condiciones claras.
RF-13	Mostrar el progreso hacia las insignias no conseguidas.
RF-14	Mostrar el resumen de los últimos siete días, agrupado por categoría.
RF-15	Editar nombre, avatar y marco desde Ajustes.
RF-16	Reiniciar todos los datos con confirmación.
RF-17	Conservar la información al cerrar y volver a abrir la app.
RF-18	Incluir datos iniciales para poder probar de inmediato.
RF-19	Programar un recordatorio local opcional por hábito, que solo avise si sigue sin marcarse.
RF-20	Generar un desafío sorpresa distinto cada día y otorgar su recompensa una única vez al cumplirse.
RF-21	Permitir comprar avatares y marcos en una tienda a cambio de monedas, sin bajar de cero.
RF-22	Mostrar adornos visuales en el avatar según el nivel alcanzado.

4.2 Requisitos no funcionales

ID	Requisito
RNF-01	Funcionamiento completamente offline; los únicos permisos declarados son opcionales y locales (recordatorios).
RNF-02	Android 7.0 (API 24) o superior.
RNF-03	Interfaz íntegramente en español.

ID	Requisito
RNF-04	Botones grandes, iconos y tarjetas; mínimo texto escrito.
RNF-05	Ningún mensaje de castigo o culpa.
RNF-06	Persistencia local real con Room, no simulada.
RNF-07	Arquitectura MVVM con separación de capas.
RNF-08	Compilación reproducible con Gradle Wrapper y JDK 17.
RNF-09	Pruebas automáticas ejecutadas en integración continua.
RNF-10	Código bajo licencia MIT.
RNF-11	Los cambios de esquema de base de datos usan migraciones reales, sin borrar datos de un héroe existente.

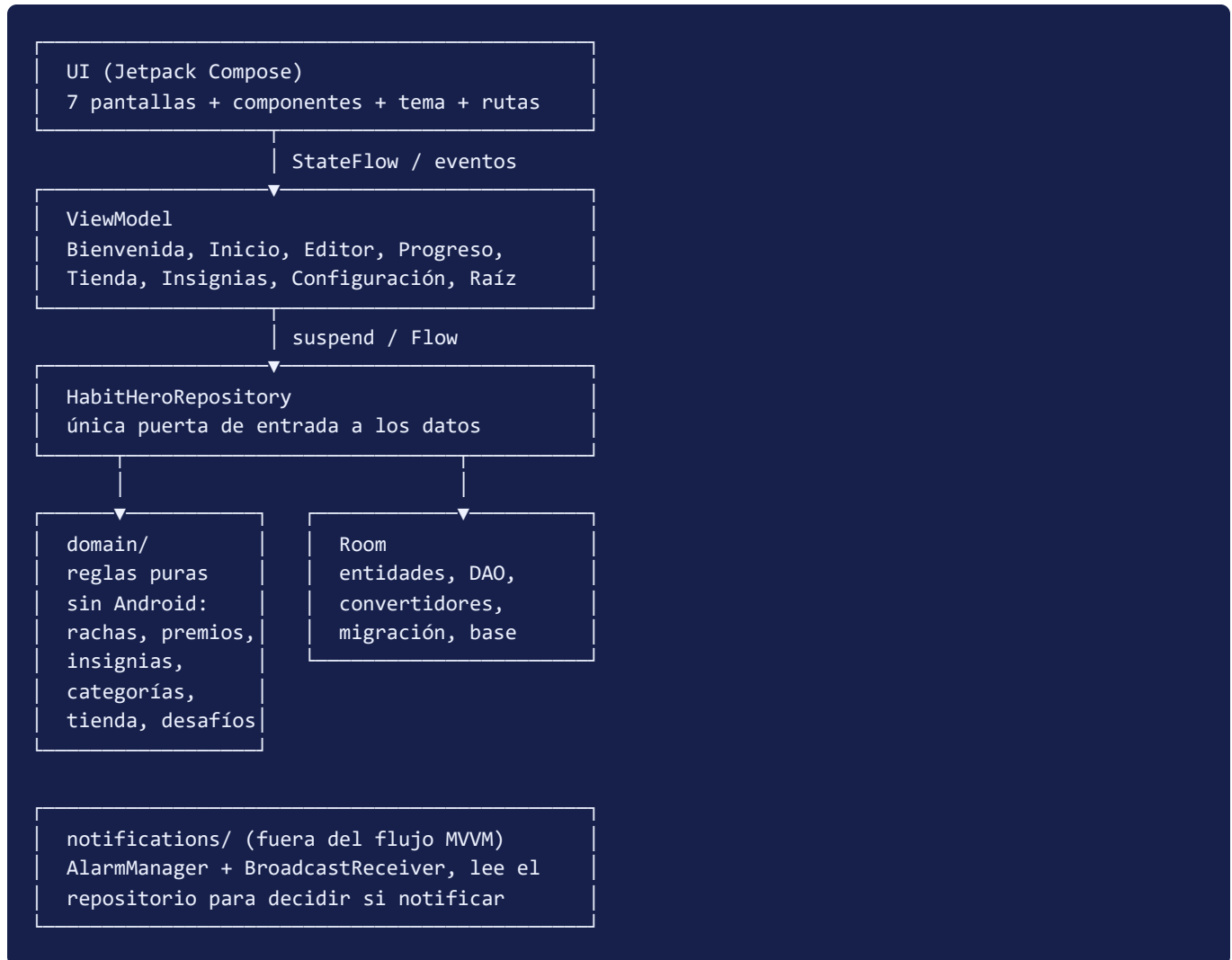
5. Pantallas

#	Pantalla	Contenido	Salidas
1	Bienvenida	Rejilla de 12 avatares, campo de nombre opcional, botón «¡Empezar!».	Inicio
2	Inicio	Cabecera con avatar (con marco y adornos de nivel), nombre, nivel, barra de experiencia y monedas. Tarjeta del desafío del día. Chips de filtro por categoría. Lista de hábitos de hoy en tarjetas grandes. Botón «Nuevo hábito».	Editor, Progreso, Insignias, Tienda, Ajustes
3	Crear o editar hábito	Vista previa en vivo, nombre con contador, 16 iconos, 6 colores, chips de categoría, selector de días con atajos, interruptor y selector de hora de recordatorio, botón guardar, borrar.	Vuelve atrás
4	Progreso	Marcas de la semana, mejor racha, porcentaje cumplido, hábitos agrupados por categoría con rejilla de 7 días cada uno.	Barra inferior
5	Tienda	Saldo de monedas, catálogo de avatares y marcos con precio y botón de compra.	Barra inferior
6	Insignias	Monedas, nivel, insignias conseguidas y barra de progreso de las pendientes.	Barra inferior
7	Ajustes	Resumen, edición de nombre, avatar y marco, reinicio con doble confirmación, información de la app.	Barra inferior

La navegación entre las cinco pantallas principales es una barra inferior con cinco destinos: Hoy, Semana, Premios, Tienda y Ajustes.

6. Arquitectura

MVVM en tres capas, con dependencias en una sola dirección. Los recordatorios locales son la única pieza que sale de este esquema, porque dependen de `AlarmManager` y de `BroadcastReceiver`s del sistema Android, no de Room ni de Compose:



Decisiones y su motivo

- **Sin librería de inyección de dependencias.** El grafo es pequeño: la clase `HabitheroApp` construye la base y el repositorio, y una única `FabricaViewModels` los entrega. Añadir Hilt habría multiplicado el andamiaje sin resolver ningún problema real.
- **Reglas de negocio en objetos puros de Kotlin** (`domain/`), sin dependencias de Android. Es lo que permite probarlas en la JVM, rápido y sin emulador; se mantuvo también para las reglas nuevas (categorías, tienda y desafíos).
- **Catálogos de contenido fijo hardcodedos, no en la base de datos.** Tanto los avatares/iconos gratuitos (`Catalogos.kt`) como el catálogo de la tienda (`TiendaCatalogo.kt`) son contenido de la app, no datos de

usuario: viven en código para no necesitar una migración cada vez que se añade un item nuevo. Lo que sí es dato real del héroe —qué compró, qué tiene equipado— se guarda en Room.

- **Recordatorios con `AlarmManager` inexacto, no `WorkManager`.** Evita sumar una dependencia nueva solo para un aviso diario, y al ser inexacto no necesita el permiso especial de alarmas exactas de Android 12+, que habría significado mandar a un niño al flujo de permisos de Ajustes del sistema.
- **Iconos como emojis en lugar de recursos gráficos.** Son coloridos, reconocibles para un niño, no pesan y eliminan una fuente entera de errores de recursos. Los marcos de la tienda siguen la misma idea: son un borde de color, no una imagen.
- **Un único esquema de color claro,** sin seguir el tema del sistema: el color es parte de la identidad de la app y no debe cambiar bajo el niño.

7. Funcionamiento

Primer arranque. Room crea `habithero.db` directamente con el esquema actual (versión 2) y ejecuta la semilla: perfil, seis hábitos con categoría, siete insignias y unas marcas de días anteriores. Como `onboardingCompletado` es 0, se abre la pantalla de bienvenida.

Uso diario. Inicio filtra los hábitos cuyo `díasSemana` contiene el día de hoy (y, si hay un filtro de categoría activo, también por categoría). Al tocar una tarjeta:

1. Se comprueba si ya hay una marca de hoy. Si la hay, no ocurre nada.
2. Se calcula la racha resultante y, con ella, monedas y experiencia.
3. Se inserta la marca. El índice único es la última barrera contra duplicados.
4. Se actualizan monedas, experiencia y nivel del perfil.
5. Se revisan las insignias y se conceden las nuevas.
6. Se revisa si el desafío del día se acaba de cumplir; si es así, se otorga su recompensa una única vez.
7. Se muestra un diálogo de celebración con lo ganado.

Tocar una tarjeta ya marcada abre la confirmación para deshacer. Al deshacer se devuelven exactamente las recompensas que dio esa marca, con suelo en 0; las insignias ya conseguidas se conservan (el desafío del día, si ya estaba completado, tampoco se revierte).

Desafío diario. Al entrar en Inicio se asegura que exista un desafío para la fecha actual: si no existe, se genera de forma determinista a partir de la fecha (la misma fecha siempre produce el mismo desafío). Se evalúa después de cada marca; una vez `completado`, no se vuelve a evaluar ni a pagar.

Tienda. Comprar un item comprueba que las monedas alcancen el precio y que no se haya comprado antes; si todo es correcto, se registra la compra y se descuentan las monedas, sin bajar nunca de 0. Los avatares y marcos comprados quedan disponibles para siempre en Ajustes.

Recordatorios. Guardar un hábito con un recordatorio activado programa una alarma diaria a esa hora; al dispararse, solo se muestra la notificación si el hábito sigue tocando hoy y aún no se ha marcado. Desactivar el recordatorio, o borrar el hábito, cancela la alarma. Si el dispositivo se reinicia, todas las alarmas activas se vuelven a programar automáticamente.

Cambio de día. Al volver a Inicio se recalcula la fecha actual, de modo que la app abierta durante la medianoche se actualiza sola (incluido el desafío del día).

8. Modelo de datos

Siete tablas: `user_profile`, `habit`, `habit_completion`, `badge`, `user_badge`, `user_unlock` y `daily_challenge`. La restricción central sigue siendo el índice único `(habitId, fecha)` sobre `habit_completion`, que traduce a nivel de base de datos la regla «un hábito solo se marca una vez por día»; `user_unlock` y `daily_challenge` usan el mismo patrón de clave primaria más `OnConflictStrategy.IGNORE` para que una compra o una evaluación repetida nunca dupliquen nada.

El esquema pasó de la versión 1 a la 2 con una migración real (`MIGRACION_1_2`, ver `docs/MANUAL_TECNICO.md` §5): columnas nuevas con `ALTER TABLE` y dos tablas nuevas, sin destruir ni perder los datos de un héroe que ya tuviera la app instalada.

El detalle completo —campos, tipos, claves, relaciones, reglas y diagrama entidad-relación en Mermaid— está en `BASE_DE_DATOS.md`.

9. Pruebas

Las pruebas viven en `app/src/test/` y se ejecutan en la JVM, sin emulador. Las que necesitan Room usan Robolectric con una base de datos en memoria (o, para la migración, un archivo real): es Room de verdad, con SQL real y las mismas restricciones.

Área	Qué se comprueba
<code>CalculadoraRachasTest</code>	Racha vacía; tres días seguidos; hoy sin marcar no rompe la racha; un día saltado sí la corta; los días no programados no cuentan; mejor racha en todo el historial.
<code>CalculadoraRecompensasTest</code>	Recompensa base; bonus a partir de 3 y de 7 días; nivel cada 100 puntos; progreso siempre entre 0 y 1; experiencia negativa no rompe nada.
<code>EvaluatorInsigniasTest</code>	Condición de primera marca; condición de racha exacta; no reentregar insignias ya conseguidas; progreso acotado a 1.
<code>EvaluatorTiendaTest</code>	Se puede comprar con monedas suficientes; no se puede comprar dos veces ni sin monedas; el saldo nunca queda negativo.
<code>GeneradorDesafiosTest</code>	La misma fecha siempre genera el mismo desafío; cualquier fecha produce un desafío de un tipo válido con recompensa positiva.
<code>EvaluatorDesafiosTest</code>	Cada uno de los tres tipos de desafío se cumple exactamente cuando corresponde (tres hábitos, todos los de hoy, antes de una hora).

Área	Qué se comprueba
MigracionTest	La migración 1→2 no lanza excepción, conserva los datos de un héroe existente (hábito, monedas, experiencia) y deja las columnas nuevas con sus valores por defecto.
HabitHeroRepositoryTest	Marcar guarda y premia; no se puede marcar dos veces el mismo día ni premiar dos veces; sí se puede marcar en días distintos; desmarcar devuelve puntos; los datos persisten al releer desde Room; borrar un hábito borra sus marcas en cascada; el nivel sube al acumular experiencia; comprar en la tienda descuenta monedas y desbloquea el item; no se puede comprar sin monedas ni repetir una compra; el desafío diario paga su recompensa una sola vez.

El flujo `android-build.yml` ejecuta `testDebugUnitTest` antes de compilar, de modo que un fallo en cualquiera de estas pruebas detiene la generación del APK.

Criterios de aceptación manual

- Las siete pantallas se abren y se vuelve atrás sin cierres inesperados.
- Al cerrar la app por completo y volver a abrirla, se conservan hábitos, marcas, monedas, nivel, insignias, compras de la tienda y el desafío del día.
- Marcar dos veces el mismo hábito el mismo día no cambia el contador de monedas.
- Activar un recordatorio, cerrar la app y esperar a la hora elegida muestra la notificación solo si el hábito sigue sin marcar.
- «Reiniciar todos los datos» deja la app como recién instalada.

10. Limitaciones

1. **APK de depuración.** El artefacto que se publica está firmado con la clave de depuración de Android. Se instala y funciona, pero no sirve para Google Play; para eso haría falta una `release` firmada con un almacén de claves propio.
2. **Un solo perfil por dispositivo.** No hay soporte para varios hermanos en la misma tableta.
3. **Recordatorios inexactos.** Al usar `AlarmManager.setRepeating` (para no pedir el permiso de alarmas exactas), el aviso puede llegar dentro de una ventana aproximada a la hora elegida, no al segundo.
4. **Sin pruebas de interfaz automatizadas.** Los tests cubren la lógica y los datos, no la navegación en Compose, que se verifica a mano.
5. **La ventana de progreso es de siete días.** No hay vista mensual ni anual.
6. **Sin exportación ni copia manual.** Los datos dependen de la copia de seguridad estándar de Android; si se desinstala la app, se pierden.
7. **El cálculo de mejor racha ignora las marcas de días no programados.** Si un hábito se edita y cambian sus días, las marcas antiguas fuera del nuevo calendario dejan de contar para la racha, aunque se conservan en la base.

8. **Catálogo de tienda y de desafíos fijos en código.** Añadir un ítem o una plantilla de desafío nueva requiere publicar una nueva versión de la app, ya que no hay un panel de administración remoto (deliberado: no hay servidor).
-

11. Conclusiones

HabitHero cumple lo que se propuso: una aplicación Android nativa, en español, completamente offline, que permite a un niño de 8 a 12 años gestionar sus hábitos diarios por sí mismo y ver su avance de un vistazo. Sobre la base ya validada (marcar, rachas, monedas, niveles, insignias), esta versión añadió cinco piezas que profundizan el ciclo de motivación sin romper ninguna de las reglas originales: recordatorios locales opcionales, categorías para organizar los hábitos, una tienda que por fin le da un uso a las monedas acumuladas, un desafío sorpresa que renueva el interés cada día, y un avatar que evoluciona visualmente con el esfuerzo del niño.

Las decisiones que más pesaron en el resultado siguen siendo las mismas tres de la primera versión, ahora extendidas a las funciones nuevas. La primera, poner las reglas de unicidad en el propio esquema de la base de datos (el índice único de las marcas, y la misma estrategia aplicada a las compras de la tienda y al desafío diario) en lugar de solo en la interfaz. La segunda, mantener las reglas de negocio en objetos puros de Kotlin, lo que permitió sumar `EvaluadorTienda`, `GeneradorDesafios` y `EvaluadorDesafios` con pruebas unitarias tan rápidas como las originales. La tercera, el compromiso de que la app nunca castigue: los recordatorios no molestan si el hábito ya está hecho, el desafío no cumplido no tiene ninguna consecuencia, y las insignias y compras nunca se retiran.

El siguiente paso natural sería una versión firmada para distribución, y después, si se confirma la necesidad, soporte para varios perfiles en el mismo dispositivo o una vista de progreso más allá de los siete días — ambos posibles sin romper el compromiso de funcionar sin conexión.